# Assignment 5: Data Compression

by Dr. Kerry Veenstra
CSE 13S, Fall 2026
Document version 1 (changes in Section 9)

Due Thursday November 12th, 2026, at 11:30 pm

## Contents

**1 Introduction** **2**
1.1 Get the Resources . . . 2
1.2 Run the Reference Programs . . . 2

**2 Make a Makefile** **4**

**3 Nibble-based File I/O** **5**
3.1 Background . . . 5
3.1.1 Example: Reading a File, Byte-by-Byte . . . 5
3.1.2 Example: Writing a File, Byte-by-Byte . . . 6
3.1.3 Isolating the Low Nibble . . . 8
3.1.4 Isolating the High Nibble . . . 9
3.1.5 Combining High and Low Nibbles . . . 10
3.1.6 Nibbler API (Application Programming Interface) . . . 11
3.2 Write and Test the Functions of the Nibbler ADT . . . 11
3.2.1 nib_open . . . 11
3.2.2 nib_close . . . 12
3.2.3 nib_get_nibble . . . 12
3.2.4 nib_put_nibble . . . 13

**4 Data Compression** **13**
4.1 Background . . . 13
4.1.1 Data Compression Algorithm . . . 14
4.2 Write and Test the Data Compressor . . . 14
4.2.1 main . . . 14
4.2.2 compress . . . 15

**5 Data Decompression** **15**
5.1 Background . . . 15
5.1.1 Data Decompression Algorithm . . . 15
5.2 Write and Test the Data Decompresser . . . 16
5.2.1 main . . . 16
5.2.2 decompress . . . 17

**6 Test Your Programs** **17**

**7 Submit: add/commit/push/submit commit ID** **18**

**8 Bonus Section: The Inconvenient Reality of Lossless Data Compression** **18**

**9 Revisions** **18**

# 1 Introduction

In this assignment you are going to write two programs: a data compressor and a corresponding decompressor.




The compression program will use a simple algorithm that is similar to the one that was used in the original Apple Macintosh *MacWrite* program to decrease the sizes of word-processing files[1].

We will lead you through a sequence of steps:

- Write a Makefile (Section 2, pages 4 – 5).
- Write an Abstract Data Type (Section 3, pages 5 – 13). The ADT will make it easier to read and write the compressed data files. You will be given unit tests for the ADT.
- Write the `compress` and `decompress` programs (Sections 4 and 5, pages 13 – 17).
- Test and submit your work (Section 6, page 17).

## 1.1 Get the Resources

☐ **To do:** In a Terminal, synchronize the **resources** repository on your Github Codespaces instance. Then copy the files for asgn5 into your repository. Remember that the you need to use the `cd` ("change directory") command to set the "current directory" of the Terminal. **The Explorer window does not affect the current directory of the Terminal.**

Note in the first `cd` command below, you want to go to `/workspaces/codespaces-blank` (notice that this "absolute path" starts with a slash `/`), but you can use "auto completion" to reduce the amount of typing that you need to do. That is, instead of typing

```
$ cd /workspaces/codespaces-blank⏎
```

(⏎ is enter on Windows or return on macOS)

You can type the first few letters of a directory name followed by a tab key, as demonstrated below.

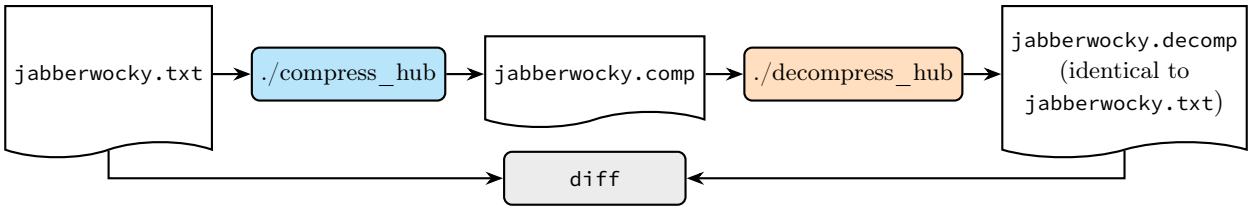
```
$ cd /w⇥/c⇥⏎
$ cd 13s/resources
$ git pull
$ cd ..
$ cp resources/asgn5/* yourcruzid/asgn5
$ cd yourcruzid/asgn5
$ ls
```

You should see six executable files (also called "reference binaries" or "demo programs"): the data compressors `compress_arm`, `compress_hub`, and `compress_x86`; and the decompressors `decompress_arm`, `decompress_hub`, and `decompress_x86`. You will be running two of these binaries depending on which microprocessor your Terminal is running on. (The next section explains this.) After you make your own version of the programs, you'll have two more binaries: `compress` and `decompress`.

## 1.2 Run the Reference Programs

Below is an example of using `compress` to reduce the size of the nonsense poem "Jabberwocky".

© 2026 Kerry Veenstra




☐ **To do:** In your repository's `asgn5` directory, you will be running a compress demo program and a decompress demo program. Most of you are using Github Codespaces, and so run the commands on the left. If you are running Ubuntu on a Mac with Apple silicon, then run the commands in the middle. Otherwise, run the commands on the right. (If you are not running on GitHub, use the selected commands instead of `./compress_hub` and `./decompress_hub` in the boxes below.)

| | | |
|---|---|---|
| `./compress_hub` | `./compress_arm` | `./compress_x86` |
| `./decompress_hub` | `./decompress_arm` | `./decompress_x86` |

```
$ ./compress_hub jabberwocky.txt -o jabberwocky.comp ⏎      ⟵ compress a file
$ ls -l jabber* ⏎      ⟵ compare the sizes of the original and compressed files
-rw-rw-r- 1 ubuntu ubuntu  814 Feb 8 14:47 jabberwocky.comp    ⟵ size of compressed file
-rw-rw-r- 1 ubuntu ubuntu 1017 Feb 8 14:46 jabberwocky.txt     ⟵ size of original file
```

The compressed file has 814 bytes, while the original file has 1017 bytes. So the compressed file is 814÷1047 = 78% of the original size.

By the way, instead of typing the entire file names `jabberwocky.txt` and `jabberwocky.comp`, you can use auto completion to reduce the amount of typing needed using the tab key and backspace key ←.

To get:

```
$ ./compress_hub jabberwocky.txt -o jabberwocky.comp Enter
```

Type:

```
$ ./compress_hub j tab -o j tab ← ← ← ← comp Enter
```

So you've created a compressed version of `jabberwocky.txt` called `jabberwocky.comp`.

☐ **To do:** The decompression program, `decompress_hub`, will reverse the compression process. Then you can compare the output of the decompressor with the original, uncompressed file. (If you are not running on GitHub, use the selected command instead of `./decompress_hub` in the box below.)

```
$ ./decompress_hub jabberwocky.comp -o jabberwocky.decomp ⏎  ⟵ decompress
$ ls -l jabber* ⏎                                           ⟵ compare file sizes
-rw-rw-r- 1 ubuntu ubuntu  814 Feb 8 14:47 jabberwocky.comp
-rw-rw-r- 1 ubuntu ubuntu 1017 Feb 8 14:53 jabberwocky.decomp  ⟵ size of decompressed file
-rw-rw-r- 1 ubuntu ubuntu 1017 Feb 8 14:46 jabberwocky.txt     ⟵ size of original file
```

The sizes of the original file `jabberwocky.txt` and the file that resulted from a compress/decompress cycle, `jabberwocky.decomp`, are the same: 1017 bytes.

Next, compare the two files using the `diff -us` command.

```
$ diff -us jabberwocky.txt jabberwocky.decomp ⏎      ⟵ compare file contents
Files jabberwocky.txt and jabberwocky.decomp are identical   ⟵ Success!
```

The `diff -us` command reports that the contents of the files are the same.

## 2 Make a Makefile

Why do you use a `Makefile`? Because you need to make three executable files: `unittests`, `compress`, and `decompress`.

Unlike some other programming languages, if you have *multiple* C program files, you create an executable file through multiple intermediate "object files" (`nibbler.o`, `unittests.o`, `compress.o`, and `decompress.o`). The steps are shown below. These are what we want the `make` command to do. (Add the `-gdwarf-4` option to the `clang -c` command to make it possible to use a debugger.)

What `make` Does when Compiling Three Executables:

```
clang -c -Wall -Werror -gdwarf-4 nibbler.c      ⟵ create nibbler.o from nibbler.c
clang -c -Wall -Werror -gdwarf-4 unittests.c    ⟵ create unittests.o from unittests.c
clang -c -Wall -Werror -gdwarf-4 compress.c     ⟵ create compress.o from compress.c
clang -c -Wall -Werror -gdwarf-4 decompress.c   ⟵ create decompress.o from decompress.c
clang unittests.o nibbler.o -o unittests        ⟵ create "executable file" unittests from
                                                  unittests.o and nibbler.o
clang compress.o nibbler.o -o compress          ⟵ create "executable file" compress from
                                                  compress.o and nibbler.o
clang decompress.o nibbler.o -o decompress      ⟵ create "executable file" decompress from
                                                  decompress.o and nibbler.o
```

☐ **To do:** Create the file `Makefile` to make the `unittests`, `compress`, and `decompress` executables.

- **The first two lines of your Makefile are below.** Remember that the `make` program will make the **first** target that is listed in your `Makefile`, along with any other dependent files. (The "target" is underlined in the dependency line below.) So if the first dependency line of a `Makefile` is the one below, then the `Makefile` will execute commands to make `unittests`, `compress`, and `decompress` (which is what you want). If the dependency for `all` is not the first dependency in the file, then typing `make` will not make all of the executables.

  When looking at a `Makefile` dependency line, remember, "this *depends on* that."

  ```
  .PHONY: all      ⟵ tell make that all isn't really a file that gets made
  all: unittests compress decompress
  ```

☐ **To do:** Add more lines to your `Makefile` so that it will compile and link `unittests`, `compress`, and `decompress`. The box above contains the various commands that need to be run. Your `Makefile` must contain appropriate dependency lines.

- Remember that Assignment 4 and the lecture on October 28th demonstrated using a `Makefile`.
- The table indicates the `target` and the `dependency` or `dependencies` of every the command. You can construct an appropriate Makefile rule for the command with this information.
- For more on Makefiles, see the library reserve book *Managing Projects with GNU Make* by Robert Mecklenburg.

☐ **To do:** Run your `Makefile` to create `unittests`, `compress`, and `decompress`. Since we've provided you with starter files for `compress.c`, `decompress.c`, `nibbler.c`, and `unittests.c`, running `make` should print no errors.

Running your `Makefile`

```
$ make      ⟵ That's all that's needed.
```

We had you make your Makefile first so that you can easily compile your C files.

# 3 Nibble-based File I/O

## 3.1 Background

All of this assignment's programs will read and write data files, and all of them will use the same set of "nibble" File I/O routines.

- This section starts with a review of the `stdio.h` library's normal character-based File I/O.
- Then it explains how you can use operators of the C programming language to access and modify individual nibbles of a byte.
- Finally it presents the "Nibbler API" (Application Programming Interface) that your data compression and data decompression programs will use.

### 3.1.1 Example: Reading a File, Byte-by-Byte

The `stdio.h` library provides functions that let a program read a file one byte at a time using three functions (one of the functions in a loop). (For more on these functions, see sections 7.5, B1, B1.1, and B1.4 of K&R.)

```
FILE *f = fopen(filename, "r");   Usually pronounced "F-open."
int ch = fgetc(f);                "F-get-C" — Call in a loop until it returns EOF.
fclose(f);                        "F-close."
```

The `fopen(filename, "r")` function is given a pointer to a string `filename` and the constant "mode" `"r"`, which means to prepare to "read" bytes from the file. (Alternatives are to prepare to "write" or "append" bytes. We will write bytes in the example after this one.)

The `fgetc()` function returns the next byte (character) of the file, or it lets you know that it has returned all of the characters of the input file by returning an integer constant `EOF` (which stands for end-of-file).

For example, if your program opens a file that contains only four characters `"abc\n"`, then five calls to the `fgetc(f)` function will return five consecutive values: `'a'`, `'b'`, `'c'`, `'\n'`, and finally the constant `EOF`.

Be sure to store the `fgetc()` return value in a variable of type `int`, *not* type `char`.

Although `fgetc()` can return $256 = 2^8$ different `char` values 0 through 255, which fit in an `unsigned char`, it also returns a $257^{\text{th}}$ value to represent `EOF`. (Usually `EOF` equals $-1$.) Since a `char` cannot store 257 different values, use an `int` variable to hold the `fgetc()` return value. (The **`int`** declaration in the code listing below is in **red** to remind you to use an **`int`**

`read_file_example.c` is an example of using these functions to read data from a file and to print it to the screen.

☐ **To do:** We've given you a C program file `read_file_example.c`. (The listing is on the next page.) Read through it.
☐ **To do:** What do you predict it will do?
☐ **To do:** Compile the program.
☐ **To do:** Run the program.
☐ **To do:** Verify that the program did what you expected.

```c
void read_file_example(const char *filename) {  ⟵ The function is called with a file name.
    /*
    * Try to open the file with the name in filename.  Complain and exit if
    * we can't open the file.
    */
    FILE *f = fopen(filename, "r");        ⟵ Open file for reading ("r"). Save return value f.

    if (f == NULL) {           ⟵ Check fopen() return value. It's NULL? Report an error.
        printf("Can't open file for input:  %s\n", filename);
        exit(1);
    }

    /*
    * Read and print chars from the file until the special end-of-file
    * value EOF is returned.  Remember that the value from fgetc() must be
    * stored in an int because a char cannot store all 256 byte values from
    * 0 to 255 as well as the special value EOF (which usually is -1).
    */
    while (1) {                ⟵ Read the characters of the file in a loop.
        int ch = fgetc(f);     ⟵ Get the next char from the file. fgetc() returns an int.
        if (ch == EOF) break;  ⟵ Leave the loop if fgetc() returns EOF ("end of file").

        // Print ch, but here is where the program could write to an output file.
        printf("%c", ch);      ⟵ Print the char.
    }

    fclose(f);                 ⟵ We are done reading the file. Close it.
}
```

### 3.1.2 Example: Writing a File, Byte-by-Byte

In addition to *reading* a data file one character (or byte) at a time, a program can use three of the `stdio.h` functions to *write* a file one byte at a time: (For more on these functions, see sections 7.5, B1, B1.1, and B1.4 of K&R.)

| | |
|---|---|
| `FILE *f = fopen(filename, "w");` | *Usually pronounced "F-open."* |
| `fputc(ch, f);` | *"F-put-C" — Call in a loop for every char to print.* |
| `fclose(f);` | *"F-close."* |

Calling `fopen(filename, "w")` will prepare a new file whose name is in the string `filename` to be written with characters from `fputc()`. If the file already exists, then calling `fopen(filename, "w")` first will erase the contents of the file. (It is possible to "append" characters to a file without first erasing its contents, but we don't do that in this assignment. See `man 3 fopen` and `"a"` mode for more information.)

`write_file_example.c` is an example of using these functions to write a character string to a file.

☐ **To do:** Read through the example C program file `write_file_example.c`. The listing is on the next page. (We talk about the variable `p` and the notation `*p` below. For more information on pointers, see Chapter 5 of K&R.)

```c
void write_file_example(const char *filename) {  ⟵ The function is called with a file name.
    /*
    * Try to create the file with the name in the parameter filename.
    * Complain and exit if we can't create the file.
    */
    FILE *f = fopen(filename, "w");        ⟵ Open file for writing ("w"). Save return value f.

    if (f == NULL) {          ⟵ Check fopen() return value. It's NULL? Report an error.
        printf("Can't open file for output:  %s\n", filename);
        exit(1);
    }

    /*
    * Here is a string of chars that we want to write to the file.
    */
    char string[] = "Write this string to the file.\n";

    /*
    * Declare a pointer p to the first char of the string.
    * Now *p == 'W' (the first char of the string).
    */
    char *p = string;

    while (*p != '\0') {      ⟵ Walk through the chars of the string.
        /*
        * The expression *p refers to the char that is pointed to by p.
        * Write that char to the file using f.
        */
        fputc(*p, f);

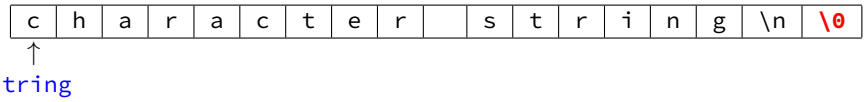
        /*
        * Point p to the next char of the string.
        */
        ++p;
    }

    fclose(f);                ⟵ We are done writing the file. Close it.
}
```

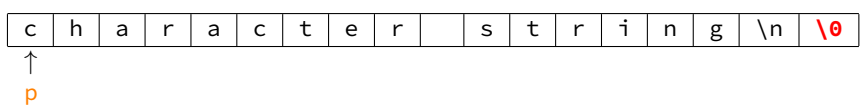
Let's talk about the `while` loop, the variable `p`, and the meaning of `*p`.

The declaration `char string[] = "character string\n";` creates

- A character array in memory containing `"character string\n\0"`, and
- A (constant) pointer named `string` to the first `char` of the array.




Then the declaration `char *p = string;` creates a (non-constant) pointer named `p` that points to the same `char` that `string` points to.



Now it's time for the loop to walk the pointer `p` along the array. Whenever you see `*p` it means the character that the pointer `p` is pointing to. (This is called "pointer dereferencing.")

```
while (*p != '\0') {        ⟵ while p doesn't point at '\0', written as *p != '\0'
    fputc(*p, f);           ⟵ Write a char of the string to the file using f.
    ++p;                    ⟵ Point to the next char in the message.
}
```

The call to `fputc(*p, f);` writes the `'c'` to the screen. After the `++p`, then `p` points at the `'h'` (meaning that `*p == 'h'`).

| c | h | a | r | a | c | t | e | r | | s | t | r | i | n | g | \n | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | ↑ | | | | | | | | | | | | | | | | |
| | p | | | | | | | | | | | | | | | | |

After another `++p`, the pointer `p` points at `'a'`, and `*p == 'a'`.

| c | h | a | r | a | c | t | e | r | | s | t | r | i | n | g | \n | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | ↑ | | | | | | | | | | | | | | | |
| | | p | | | | | | | | | | | | | | | |

Eventually `p` will point at `'\0'`, meaning `*p == '\0'`, ...

| c | h | a | r | a | c | t | e | r | | s | t | r | i | n | g | \n | \0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | | | | | | | ↑ |
| | | | | | | | | | | | | | | | | | p |

...and then the condition of `while (*p != '\0')` will become `false`, meaning the loop ends. Using a `while()` loop and a pointer to walk through an array is very common in C.

☐ **To do:** Read through the example C program file `write_file_example.c` again.
☐ **To do:** What do you predict it will do?
☐ **To do:** Compile the program.
☐ **To do:** Run the program.
☐ **To do:** The program creates a file `out.txt`. Dump the file in hexadecimal using the `xxd` command.
☐ **To do:** Did the program in `write_file_example.c` do what you thought it would?

You have seen how a program can read and write files one 8-bit byte at a time.

Instead of writing full 8-bit bytes, the data compression algorithm that will be presented in Section 4 writes 4-bit chunks of data. Let's talk about that.

### 3.1.3 Isolating the Low Nibble

The computer-science term for a 4-bit chunk of data is "nibble." The $16 = 2^4$ possible values of a nibble range from binary 0000 through binary 1111 (decimal 0 through decimal 15). The eight bits of a byte can be split into two nibbles, often called the "high nibble" and the "low nibble."

| High Nibble | | | | Low Nibble | | | | |
|---|---|---|---|---|---|---|---|---|
| $b_7$ | $b_6$ | $b_5$ | $b_4$ | $b_3$ | $b_2$ | $b_1$ | $b_0$ | ⟵ 8-bit value $b_7b_6b_5b_4b_3b_2b_1b_0$ |
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | ⟵ bit number of the byte |

In the figure above, the Low Nibble occupies the four least-significant bits of a `char` ($b_3b_2b_1b_0$). You can use the & (bit-wise-*and*) operator to isolate the four least-significant bits of a `char` by bit-wise *and*ing with a constant value of binary $00001111_2$. (For more on the `&` operator, see sections 2.9 and A7.11 of K&R.)

(in binary) $b_7b_6b_5b_4b_3b_2b_1b_0$ `&` `00001111` ⟶ `0000`$b_3b_2b_1b_0$

(in C) `low_nibble = my_byte & 0b00001111;`

Notice how C supports directly entering a binary number by putting "`0b`" in front. `0b00001111`.

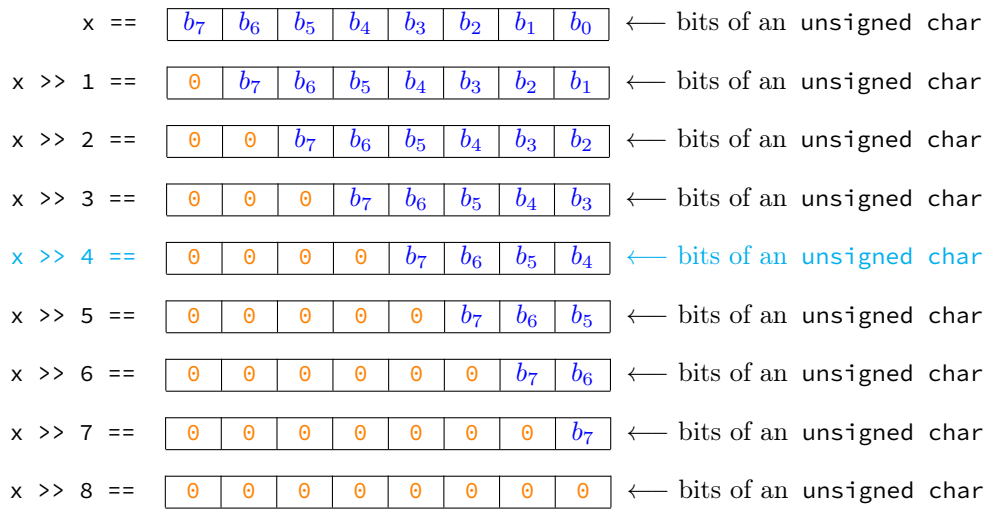
### 3.1.4 Isolating the High Nibble

The C programming language has an operator that you can use to isolate the high nibble of a byte by shifting bits right: `>>`. (For more on the `>>` operator, see sections 2.9 and A7.8 of K&R.)

**`unsigned char >> num_bits`**

The `>>` operator shifts bits of an integer expression to the right by the given number of bits. If the expression is an `unsigned char`, then the `>>` operator shifts in `0` bits from the left.

You can use `x >> 4` to isolate the high (left) nibble ($b_7b_6b_5b_4$) of a `unsigned char`:




Isolating the high nibble is similar, but first you would shift the high nibble to the right by four bit positions. Then you would isolate the bits of the low nibble.

(in binary) `(`$b_7b_6b_5b_4b_3b_2b_1b_0$ `>> 4) & 00001111` ⟶ `0000`$b_7b_6b_5b_4$

(in C) `high_nibble = (my_byte >> 4) & 0b00001111;`

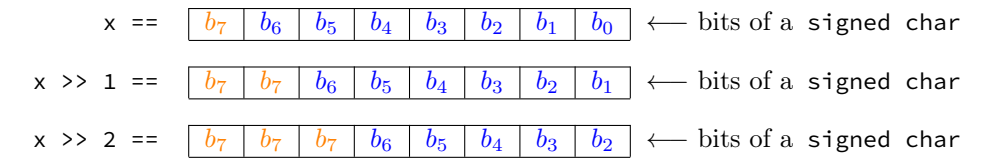
The `clang` C compiler in GitHub Codespaces treats an ordinary `char` as a `signed char` (see below). For safety, you should declare any `char` that is going to be shifted using `>>` as an `unsigned char`.

**`signed char >> num_bits` (Unusual)**

Normally one doesn't shift a `signed char` to the right, but let's talk about it because our `clang` compiler treats a `char` like a `signed char`[1].

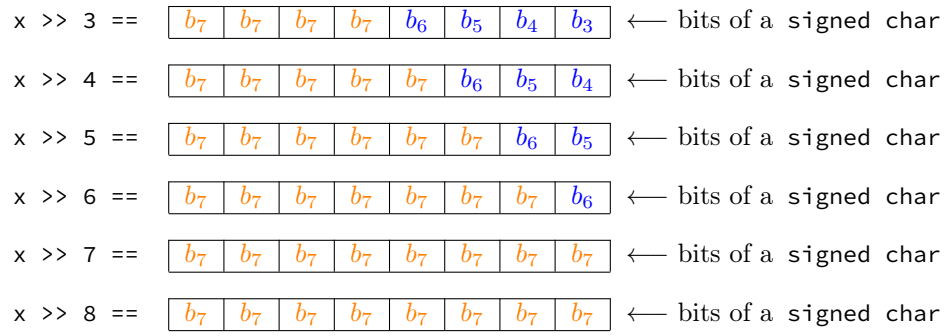
Shifting a `signed char` to the right using `>>` gives a different result than shifting an `unsigned char` to the right. A signed char duplicates its sign bit ($b_7$) to avoid changing the sign of its value. That is, if the sign bit $b_7$ is 1, then the value is negative, and to ensure that the shifted value also is negative, the sign bit is duplicated. A similar argument goes when sign bit is 0. (In assembly language, this operation often is called an "arithmetic shift.") So, to prevent the sign from changing, the sign bit $b_7$ stays in the sign bit position.

(Shifting a `signed char` usually is not done, but here's what it looks like.)




[1] I want to avoid overcomplicating things, but if you are curious about checking whether a C compiler treats a `char` as signed or unsigned, you can `#include <limits.h>`, and then use a call to `printf("%d..%d", CHAR_MIN, CHAR_MAX)` to get the values of `CHAR_MIN` and `CHAR_MAX`. If the result reports that the range of values for a `char` is `0..255`, then the C compiler treats a `char` as an `unsigned char`. If the result reports that the range of values for a `char` is `-128..127`, then the C compiler treats a `char` as a `signed char`. The easiest approach is to declare `chars` as `signed` or `unsigned`, but if you want your code to know the "signedness" of a `char`, you can use an `if` statement that checks for `CHAR_MIN == 0`.

© 2026 Kerry Veenstra




Again, normally we wouldn't shift a `signed char` using `>>`.

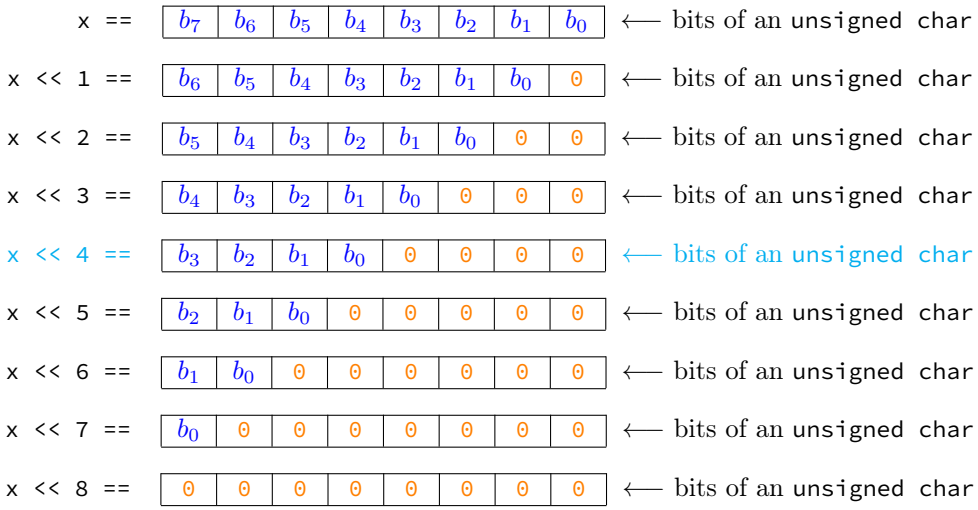
### 3.1.5 Combining High and Low Nibbles

The C programming language has two operators that you can use to combine high and low nibbles into a byte by shifting bits left (`<<` operator) and bit-wise-*or*ing (`|`). (For more on the `>>` operator, see sections 2.9 and A7.8 of K&R. For more on the `|` operator, see sections 2.9 and A7.13 of K&R.)

**`unsigned char << num_bits`**

The `<<` operator shifts bits of an integer expression to the left by the given number of bits. For example, given a byte value `x =` *8-bit-value* $b_7b_6b_5b_4b_3b_2b_1b_0$, the results of shifting to the left by 1 through 8 bits are below. Notice that `<<` always shifts in `0` bits from the right.

You can use `x << 4` to move a 4-bit value $b_3b_2b_1b_0$ into the high (left) nibble of a byte or char:




You can combine a high nibble and a low nibble into a single byte value. That is, take one value from 0 to 15 (binary `0000`$b_7b_6b_5b_4$) and merge it to the left of a second value from 0 to 15 (binary `0000`$b_3b_2b_1b_0$). The result is an `unsigned char` with the value of binary $b_7b_6b_5b_4b_3b_2b_1b_0$.

(in binary) `(0000`$b_7b_6b_5b_4$ `<< 4) | 0000`$b_3b_2b_1b_0$ $\longrightarrow$ $b_7b_6b_5b_4b_3b_2b_1b_0$

(in C) `my_byte = (high_nibble << 4) | low_nibble;`

Next we will use these operators (`<<`, `>>`, `&`, and `|`) in functions that will read and write nibbles.

### 3.1.6 Nibbler API (Application Programming Interface)

The file `nibbler.h` defines a struct and four functions that you will use to read and create files one nibble at a time. The struct, called `NIB`, manages the reading and writing of nibbles. You use `NIB` for nibble-based file I/O very much as you use `FILE` for character-based file I/O (Sections 3.1.1 and 3.1.2). The table below compares the functions of `stdio.h` and the functions of `nibbler.h`. You will be writing the functions that are defined in `nibbler.h`.

| | **Byte I/O** | ⟶ | **Nibble I/O** |
|---|---|---|---|
| Include File | `#include <stdio.h>` | ⟶ | `#include "nibbler.h"` |
| Data Declaration | `FILE *f;` | ⟶ | `NIB *nib;` |
| Opening a File | `f = fopen(filename, mode);` | ⟶ | `nib = nib_open(filename, mode);` |
| Reading Data | `char ch = fgetc(f);` | ⟶ | `int nibble = nib_get_nibble(nib);` |
| Writing Data | `fputc(ch, f);` | ⟶ | `nib_put_nibble(nibble, nib);` |
| Closing a File | `fclose(f);` | ⟶ | `nib_close(nib);` |

Internally, the Nibble-I/O functions in the table above will call traditional Byte-I/O functions as needed. They will use the `NIB` data type defined below to guide their operation. For example:

- `nib_open()` will allocate a `NIB` data structure, call `fopen()`, and then initialize fields of the `NIB` data structure. See Section 3.2.1.
- `nib_close()` will close the file by calling `fclose()`. However, if the `NIB` data structure is storing a nibble, then before closing the file, `nib_close()` will write that nibble to the file as the most-significant nibble of a `char` (whose least-significant nibble is $1111_2$). See Section 3.2.2.
- `nib_get_nibble()` will read a `char` using `fgetc()`, **store** the **least-significant nibble** in the `NIB` struct (as an integer value 0 through 15), and **return** the **most-significant nibble** as an integer value 0 through 15. However, if `nib_get_nibble()` already has a stored nibble (from an earlier call), then instead of calling `fgetc()` it will return the stored nibble. See Section 3.2.3.
- `nib_put_nibble()` will store a nibble in `nib->stored_nibble`, set `nib->num_nibbles = 1`, and wait to be called with a second nibble. Then it will merge the two nibbles into a `char` that it writes using a call to `fputc()`. After which it will set `nib->num_nibbles = 0`. See Section 3.2.4.

```
typedef struct nib NIB;

struct nib {
    FILE *underlying_f;     // The result of calling fopen().
    bool opened_for_read;   // true when fopen()'s mode was "r".  false otherwise.
    int num_nibbles;        // 0 or 1
    int stored_nibble;      // Use the value when num_nibbles == 1.
};
```

## 3.2 Write and Test the Functions of the Nibbler ADT

The descriptions of the four ADT functions are below. Also, look in `nibbler.h` and `nibbler.c` as well as Section 3.

### 3.2.1 nib_open

The `nib_open()` function opens a file so that it can be used by `nib_get_nibble()` or `nib_put_nibble()`. Here is its pseudocode:

```
nib_open(filename, mode):
    Using fopen(), open file "filename" with mode "mode".

    if fopen() returns NULL then
        return NULL

    Allocate a NIB struct using calloc().

    Initialize the NIB struct:
        nib->underlying_f holds the return value from fopen().
        nib->opened_for_read is set to 1 if mode starts with 'r'.

    Return a pointer to the NIB struct.
```

☐ **To do:** Write a function-header comment for the function `nib_open()`. Remember to include these parts:

- **Purpose**: What is the function's purpose?
- **Parameters**: What are the names, types, and uses of the functions input parameters? (Or if appropriate, mention that there are none.)
- **Returns**: What is the type and purpose of the function's return value? (Or if appropriate, mention that there is none.)

☐ **To do:** Write `nib_open()`.
☐ **To do:** Confirm that the first three unit tests pass when you run `./unittests 3`.

If a test fails, then you will need to open `unittests.c` and look at the code on and near the offending line.

### 3.2.2 nib_close

The `nib_close()` function closes a file that originally had been opened by `nib_open()`. Here is its pseudocode:

```
nib_close(nib):
    If the file is opened for write and nib->num_nibbles > 0 then:
        Create a byte with
            high nibble = nib->stored_nibble
            low nibble = 0b1111
        Write the byte using fputc(byte, nib->underlying_f).
    Close the file with fclose(nib->underlying_f).
    Free the NIB struct using free().
```

☐ **To do:** Write a function-header comment for the function `nib_close()`.
☐ **To do:** Write `nib_close()`.
☐ **To do:** Confirm that the first five unit tests pass when you run `./unittests 5`.

### 3.2.3 nib_get_nibble

The `nib_get_nibble()` function reads a nibble from an open input file. Here is its pseudocode:

```
nib_get_nibble(nib):
    if nib->num_nibbles > 0:
        nib->num_nibbles = 0
        Return nib->stored_nibble.
    else
        int byte = fgetc(nib->underlying_f)
        If byte is EOF then return EOF.
        nib->stored_nibble = low nibble of the byte.
        nib->num_nibbles = 1
        Return high nibble of the byte.
```

☐ **To do:** Write a function-header comment for the function `nib_get_nibble()`.
☐ **To do:** Write `nib_get_nibble()`.
☐ **To do:** Confirm that the first eight unit tests pass when you run `./unittests 8`.

#### 3.2.4 nib_put_nibble

The `nib_put_nibble()` function writes a nibble to an open output file. Here is its pseudocode:

```
nib_put_nibble(nibble, nib):
    if nib->num_nibbles > 0:
        Create a byte using nib->stored_nibble as the high nibble and the
           function's parameter "nibble" as the low nibble.
        Write the byte using fputc(byte, nib->underlying_f).
        nib->num_nibbles = 0
    else
        nib->stored_nibble = nibble
        nib->num_nibbles = 1
```

☐ **To do:** Write a function-header comment for the function `nib_put_nibble()`.
☐ **To do:** Write `nib_put_nibble()`.
☐ **To do:** Confirm that all 12 unit tests pass when you run `./unittests 12`.

# 4 Data Compression

## 4.1 Background

The characters of normal text files all have the same size (1 byte). To compress such files, your compression program relies on some characters being more common than others. Each common character is converted into 1 nibble, while less common characters are converted into 3 nibbles. On average, when compressing modern English text, the compressed file will be smaller than the uncompressed file.

The 15 specific characters in the list below are considered to be "common." Each of these characters will be represented in a compressed-data file by a different nibble (see Fig. 2). Everything else will be represented by a unique sequence of 3 nibbles:

a, c, d, e, f, h, i, l, n, o, p, r, s, t, *space* → 1 nibble
all others → 3 nibbles

Consider the 16-character phrase "computer science". Each of the 14 red underlined characters is converted using the table in Fig. 2. For example, `'o'` → 5 = $5_{16}$. The two characters that are not in the table are represented by their ASCII hex value with a $f_{16}$ prefix: m → ASCII $6d_{16}$ → $f6d_{16}$ and u → ASCII

$75_{16} \rightarrow f75_{16}$. The resulting nibble sequence (in hexadecimal) is c5f6def7521408c713c1. The total size of the compressed data is:

$$14 \times 1 \text{ nibble} + 2 \times 3 \text{ nibbles} = 20 \text{ nibbles} = 10 \text{ bytes}$$

The size of the compressed data is $10 \div 16 \approx 63\%$ of the original data, for an overall reduction in space.

### 4.1.1 Data Compression Algorithm

As described above, the *MacWrite* word processor reduced 15 of the possible byte values to 1 nibble, and it expanded the remaining 241 byte values to 3 nibbles. A summary of our version of the algorithm is

1. Use `fgetc()` to read a byte of data from the input file.
2. Figure out whether the byte will be represented by one nibble or by three nibbles.
3. Use the `nibbler` ADT in Section 3 to write one nibble or three nibbles to the output file.
4. Repeat steps 1–3 for all remaining bytes of the input file.

See the details of the algorithm in Fig. 1.

```
compress():
    loop:
        Use fgetc() to get an int ch.
        If ch is EOF then break
        If ch is in the table in Fig. 2.
            Convert ch into its corresponding nibble from Fig. 2.
            Write the nibble using nib_put_nibble().
        else
            Write a 15 nibble using nib_put_nibble()
            Write the high nibble of ch using nib_put_nibble()
            Write the low nibble of ch using nib_put_nibble()
```

Figure 1: `compress()` algorithm. Uses table in Fig. 2.

| | | |
|---|---|---|
| `' '` $\longrightarrow 0 = 0_{16} = 0000_2$ | `'o'` $\longrightarrow 5 = 5_{16} = 0101_2$ | `'l'` $\longrightarrow 10 = A_{16} = 1010_2$ |
| `'e'` $\longrightarrow 1 = 1_{16} = 0001_2$ | `'a'` $\longrightarrow 6 = 6_{16} = 0110_2$ | `'h'` $\longrightarrow 11 = B_{16} = 1011_2$ |
| `'t'` $\longrightarrow 2 = 2_{16} = 0010_2$ | `'i'` $\longrightarrow 7 = 7_{16} = 0111_2$ | `'c'` $\longrightarrow 12 = C_{16} = 1100_2$ |
| `'n'` $\longrightarrow 3 = 3_{16} = 0011_2$ | `'s'` $\longrightarrow 8 = 8_{16} = 1000_2$ | `'f'` $\longrightarrow 13 = D_{16} = 1101_2$ |
| `'r'` $\longrightarrow 4 = 4_{16} = 0100_2$ | `'d'` $\longrightarrow 9 = 9_{16} = 1001_2$ | `'p'` $\longrightarrow 14 = E_{16} = 1110_2$ |

Figure 2: Lookup table for converting a byte into a nibble. Used by the `compress()` algorithm of Fig. 1.

## 4.2 Write and Test the Data Compressor

Your `Makefile` will make a program called `compress` by compiling `compress.c`.

### 4.2.1 main

The `main()` function confirms that the command line has these arguments after the program name: "infile `-o` outfile". Then it will open the infile using `fopen()` and open the outfile using `nib_open()` and call `compress()`. Here is its pseudocode:

```
main(argc, argv):
    if argc != 4 then call print_usage().
    if argv[2] is not "-o" then call print_usage().
        // Note: argv[2] != "-o" will not work. Why? What should you do instead?
    Treat argv[1] as the name of the input file.
    Treat argv[3] as the name of the output file.

    Open the input file using fopen().
        if unable to open the input file then:
            print error "Can't open file for input: FILENAME\n"
            exit(1)

    Open the output file using nib_open(), and report any error.
        if unable to open the output file then:
            print error "Can't open file for output: FILENAME\n"
            exit(1)

    Call compress().
    Close the files.
```

☐ **To do:** Write a function-header comment for the function `main()`.
☐ **To do:** Write `main()`.

#### 4.2.2 compress

The `compress(fin, nib)` function compresses data from input file `fin` and writes the compressed data to output file `nib`. It assumes that `fin` already has been opened by `fin = fopen(infilename, "r")` and that `nib` already has been opened by `nib = nib_open(outfilename, "w")`.

Data compression is documented in Section 4.1.1.

☐ **To do:** Comments! Write a Function Header Comment for the `compress()` function.
☐ **To do:** Write `compress()`. The algorithm for `compress()` is documented in Fig. 1 and Fig. 2 of Section 4.1.1.
☐ **To do:** Be sure that you've included useful comments in the function body.

# 5 Data Decompression

## 5.1 Background

### 5.1.1 Data Decompression Algorithm

Decompression is the reverse of compression. Our summary of the decompression algorithm is

1. Use the `nibbler` ADT in Section 3 to read a nibble from input the file.
2. If the nibble is `EOF`, then reading the input file has ended. If the nibble is between 0 and 14, then the nibble represents a byte. Convert the nibble into that byte. Otherwise, the nibble is 15. Read the next two nibbles, and combine them to make a byte. **If a 15 nibble is read and either of the two nibbles that follow is `EOF` then the input file has ended, and your program should ignore the 15 nibble and any nibble that followed.**
3. Use `fputc()` to write the byte of data to the output file.
4. Repeat steps 1–3 for all remaining data of the input file.

```
decompress():
   loop
      read a nibble using nib_get_nibble()
      if nibble is EOF then break
      if nibble == 15:
         read another nibble as nibble1 using nib_get_nibble()
         if nibble1 is EOF then break
         read another nibble as nibble2 using nib_get_nibble()
         if nibble2 is EOF then break
         create a byte using nibble1 as the high nibble and nibble2 as the low nibble
         output the byte using fputc()
      else
         Look up nibble in the table of Fig. 4
         output the corresponding byte value
```

Figure 3: `decompress()` algorithm. Uses table in Fig. 4.

| | | |
|---|---|---|
| $0000_2 = 0_{16} = 0 \longrightarrow$ `' '` | $0101_2 = 5_{16} = 5 \longrightarrow$ `'o'` | $1010_2 = \mathrm{A}_{16} = 10 \longrightarrow$ `'l'` |
| $0001_2 = 1_{16} = 1 \longrightarrow$ `'e'` | $0110_2 = 6_{16} = 6 \longrightarrow$ `'a'` | $1011_2 = \mathrm{B}_{16} = 11 \longrightarrow$ `'h'` |
| $0010_2 = 2_{16} = 2 \longrightarrow$ `'t'` | $0111_2 = 7_{16} = 7 \longrightarrow$ `'i'` | $1100_2 = \mathrm{C}_{16} = 12 \longrightarrow$ `'c'` |
| $0011_2 = 3_{16} = 3 \longrightarrow$ `'n'` | $1000_2 = 8_{16} = 8 \longrightarrow$ `'s'` | $1101_2 = \mathrm{D}_{16} = 13 \longrightarrow$ `'f'` |
| $0100_2 = 4_{16} = 4 \longrightarrow$ `'r'` | $1001_2 = 9_{16} = 9 \longrightarrow$ `'d'` | $1110_2 = \mathrm{E}_{16} = 14 \longrightarrow$ `'p'` |

Figure 4: Lookup table for converting a nibble into a byte. Used by the `decompress()` algorithm of Fig. 3. You can implement this table in C using a `switch()` statement or an array.

See the details of the algorithm in Fig. 3.
For example, to decompress the nibble sequence `c5f6def75` `21408c713c1`, Start by looking up individual, non-$\mathtt{f}_{16}$ nibbles in the table of Fig. 4. The first nibble $\mathtt{c}_{16} = 12_{10} \rightarrow$ `'c'` and the second nibble $\mathtt{5}_{16} = 5_{10} \rightarrow$ `'o'`. Next is an $\mathtt{f}_{16}$ nibble, which identifies the first nibble of a three-nibble sequence: `f 6 d`. Convert these nibbles into a byte by throwing away the $\mathtt{f}_{16}$ and combining each remaining pair of nibbles: `f 6 d` $\rightarrow \mathtt{6d}_{16} \rightarrow$ `'m'`. Finish the conversion, and the decompressed result is `computer science`.

## 5.2 Write and Test the Data Decompresser

Your `Makefile` will make a second program called `decompress` by compiling `decompress.c`.

### 5.2.1 main

The `main()` function confirms that the command line has these arguments after the program name: "infile `-o` outfile". Then it will open the infile using `nib_open()`, open the outfile using `fopen()`, and call `decompress()`. Here is the pseudocode:

```
    main(argc, argv)
        if argc != 4 then call print_usage().
        if argv[2] is not "-o" then call print_usage().
            // Note: argv[2] != "-o" will not work. Why? What should you do instead?
        Treat argv[1] as the name of the input file.
        Treat argv[3] as the name of the output file.

        Open the input file using nib_open().
            if unable to open the input file then:
```

```
            print error "Can't open file for input: FILENAME\n"
            exit(1)

    Open the output file using fopen(), and report any error.
        if unable to open the output file then:
            print error "Can't open file for output: FILENAME\n"
            exit(1)

    Call decompress().
    Close the files.
```

☐ **To do:** Write a Function Header Comment for the `main()` function.
☐ **To do:** Write `main()`.

### 5.2.2 decompress

The `decompress(nib, fout)` function decompresses data from input file `nib` and writes the decompressed data to output file `fout`. It assumes that `nib` already has been opened by `nib = nib_open(infilename, "r")` and that `fout` already has been opened by `fout = fopen(outfilename, "w")`.

Data decompression is documented in Section 5.1.1.

☐ **To do:** Comments! Write a Function Header Comment for the `decompress()` function.
☐ **To do:** Write `decompress()`. The algorithm for `decompress()` is documented in Fig. 3 and Fig. 4 of Section 5.1.1.
☐ **To do:** Be sure that you've included useful comments in the function body.

# 6 Test Your Programs

To receive full credit, the output of your programs must match the output of the reference programs that we provide. You can use the `diff` program as you did in prior assignments to compare the output of the reference programs with the output of your programs.

Here is an example of testing both `compress` and `decompress`. The first command of the test creates the expected compressed file `compress.expect` from the reference program. That file then is used for the rest of the test.

Testing `compress` and `decompress` in the Terminal

```
$ ./compress_hub jabberwocky.txt -o compress.expect   ⟵ expected compressed data
$ ./compress jabberwocky.txt -o compress.got          ⟵ run your compress program
$ diff -us compress.expect compress.got               ⟵ compare compress results
$ ./decompress compress.expect -o decompress.got      ⟵ run your decompress program
$ diff -us jabberwocky.txt decompress.got             ⟵ compare decompress results
```

You can repeat this test with other input files besides `jabberwocky.txt`.

## 7 Submit: add/commit/push/submit commit ID

You need to submit these files. (Did you follow the instructions above to comment the two `main()` functions, `compress()`, and `decompress()`?)

- `Makefile`
- `compress.c`
- `decompress.c`
- `nibbler.c`
- `unittests.c`
- `nibbler.h`

☐ **To do:** `git add` all of the files above.
☐ **To do:** `git commit -m '`*the changes that you made*`'` (you can give any reason for this commit)
☐ **To do:** `git push`
☐ **To do:** Run the `asgn5` pipeline at **git.ucsc.edu**.
☐ **To do:** Submit your final commit ID to Canvas by Thursday November 12th at 11:30 pm.

## 8 Bonus Section: The Inconvenient Reality of Lossless Data Compression

Most files of modern English text will compress well using this algorithm, but that's not a guarantee. In fact, for any *lossless* data-compression algorithm, we *always* can create a data file that does not compress—and actually expands[2]. For our algorithm, consider the 5-letter word "buggy". Your compression program considers none of the letters to be "common," and so each will be represented by 3 nibbles. The original 10-nibble word will grow to 15 nibbles: `f62f75f67f67f79`.

$$5 \times 3 \text{ nibbles} = 15 \text{ nibbles} = 7\tfrac{1}{2} \text{ bytes}$$

The size of the "compressed" data is $7\frac{1}{2} \div 5 = 150\%$ of the original data, or actually an expansion.

Every data-compression algorithm is designed for the kind of data that it compresses. Consequently, there are as many data-compression algorithms as there are kinds of data. (The book *Data Compression: The Complete Reference, 4th ed.* [1] has over 1000 pages!) For this reason, we will be testing your programs using English-language text files—the kind of data that the *MacWrite* compression algorithm was designed for. But since it will be interesting, we may throw in a few non-English text files to see how the algorithm works.

## 9 Revisions

**Version 1** Original.

## References


[1] David Salomon. *Data Compression: The Complete Reference, 4th ed.*, pages 20–21. Springer-Verlag, London Ltd., 4th edition, 2007. (Or pages 18–19 in 3rd edition, 2004.).

[2] Wikipedia. Lossless compression — Wikipedia, the free encyclopedia, 2025. [Online; accessed 13-Feb-2025]. URL: `https://en.wikipedia.org/wiki/Lossless_compression#Limitations`.


© 2026 Kerry Veenstra